

OPERATING SYSTEMS
UE25CS242A

Classic problems of Synchronization

Dr. Anitha K
Department of Computer Science and Engineering

Acknowledgement, Prof. Pavan A C

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:

1. “Operating System Concepts”, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. “Advanced Programming in the Unix Environment”, Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from “Operating Systems, Internals and Design Principles”, William Stallings, 9th Edition, Pearson, 2018.

OPERATING SYSTEMS

Classical problems of Synchronization

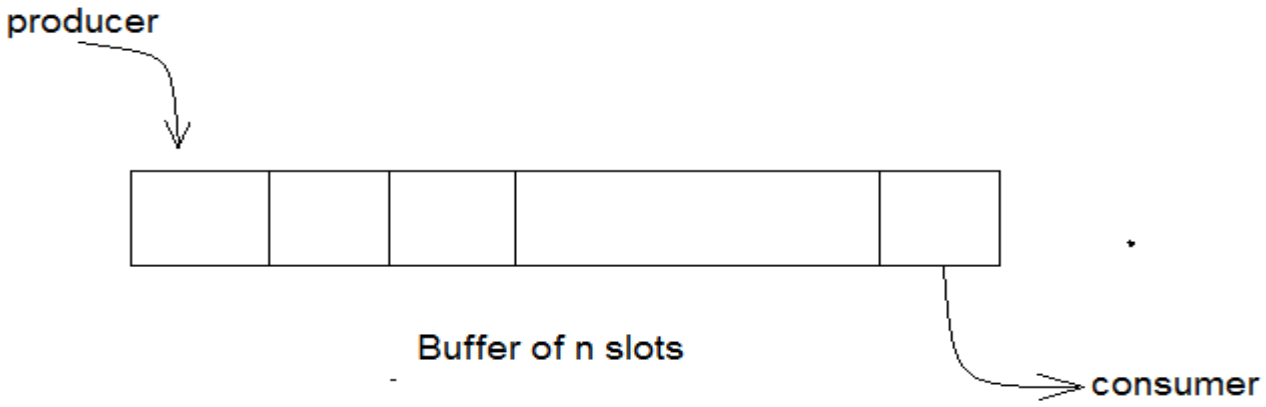
Dr. Anitha K

Department of Computer Science and Engineering

OPERATING SYSTEMS

Bounded-Buffer Problem

- ***n*** buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n



Bounded-Buffer Problem (Cont.)



- The structure of the producer process

```
do {
    ...
    /* produce an item in next_produced */
    ...
    wait(empty); //wait until empty > 0 and then decrement 'empty'
    wait(mutex); //acquire lock
    ...
    /* add next produced to the buffer */
    ...
    signal(mutex); //release a lock
    signal(full); //increment full
} while (true);
```

Bounded-Buffer Problem (Cont.)



- The structure of the consumer process

```
do {
  wait(full); // wait until full > 0 and then decrement 'full'
  wait(mutex); // acquire the lock
  ...
  /* remove an item from buffer to next_consumed */
  ...
  signal(mutex); // release the lock
  signal(empty); // increment 'empty'
  ...
  /* consume the item in next consumed */
  ...
} while (true);
```

Readers-Writers Problem



The Problem Statement

There is a shared resource which should be accessed by multiple processes. There are two types of processes in this context. They are **reader** and **writer**. Any number of **readers** can read from the shared resource simultaneously, but only one **writer** can write to the shared resource. When a **writer** is writing data to the resource, no other process can access the resource. A **writer** cannot write to the resource if there are non zero number of readers accessing the resource at that time.

Readers-Writers Problem



Solution

- Several variations of how readers and writers are considered – all involve some form of priorities
- Shared Data
  - Data set
  - Semaphore **rw_mutex** initialized to 1(semaphore)
  - Semaphore **mutex** initialized to 1 (mutex)
  - Integer **read_count** initialized to 0

Readers-Writers Problem (Cont.)



- The structure of a writer process

```
do {
        wait(rw_mutex);
        ...
        /* writing is performed */
        ...
        signal(rw_mutex);
} while (true);
```

Readers-Writers Problem (Cont.)



- The structure of a reader process

```
do {
        wait(mutex);
        read_count++;
        if (read_count == 1)
            wait(rw_mutex);
        signal(mutex);
        ...
        /* reading is performed */
        ...
        wait(mutex);
        read count--;
        if (read_count == 0)
            signal(rw_mutex);
        signal(mutex);
} while (true);
```

Readers-Writers Problem Variations



- ***First*** variation – no reader kept waiting unless writer has permission to use shared object.
- ***Second*** variation – once writer is ready, it performs the write ASAP.
- Both may have starvation leading to even more variations.
- Problem is solved on some systems by kernel providing reader-writer locks.

Real life examples



- *Database Management Systems (DBMS): Most databases are "read-mostly." Thousands of users might query a product's price (readers), but the store manager only updates it once a day (writer). Blocking all readers during every read would be inefficient, but allowing a read during an update could result in a customer seeing a corrupted or partial price.*

- *File Systems: Multiple processes might read a configuration file or a log. If a system update needs to change that configuration, it must wait until the readers are finished to ensure the file isn't left in an inconsistent state.*

- *In-Memory Caches: Systems like Redis or internal application caches often serve many threads. When the cache needs to be invalidated or updated, the Readers-Writers pattern ensures that no thread reads "stale" or partially updated data.*

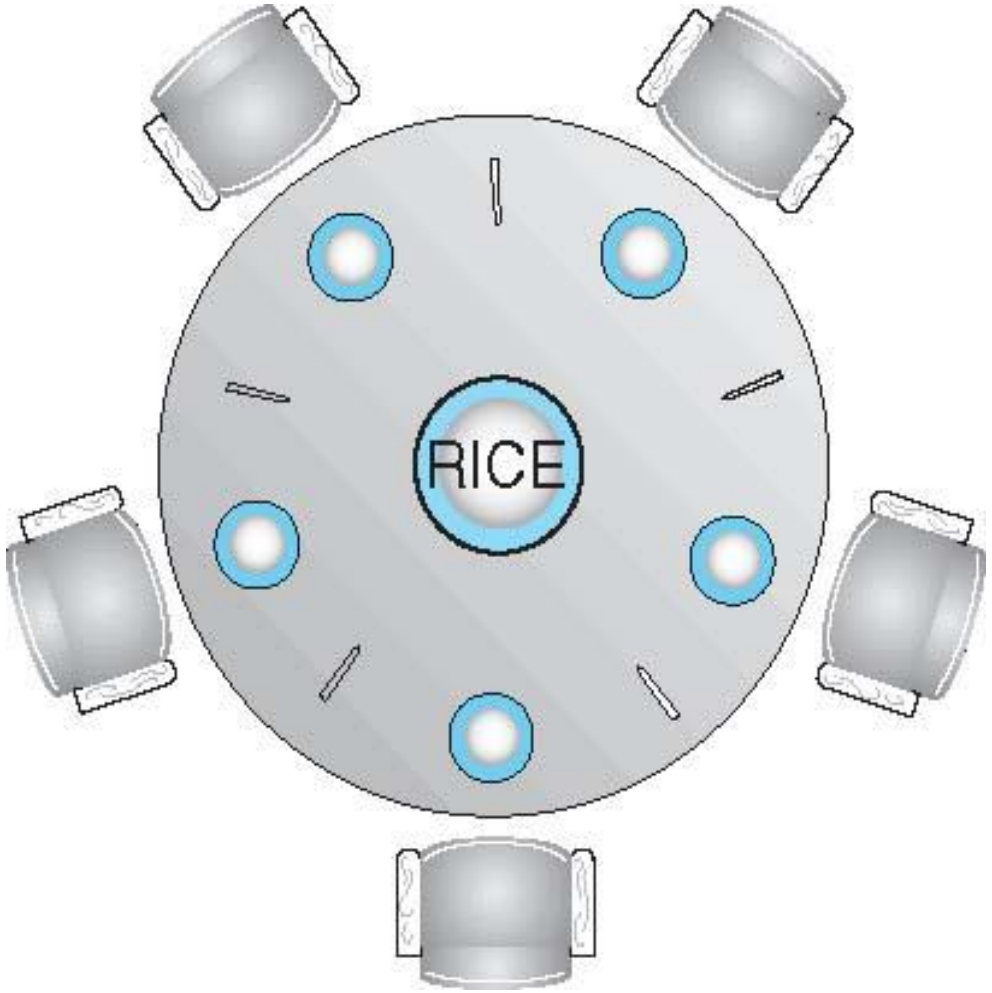
Linux Specifics



- *rwlock_t: In the Linux kernel, the Readers-Writers problem is so common that there is a dedicated lock type. It allows multiple CPUs to hold the lock for reading but only one for writing.*
- *This has Write starvation problem as it is biased towards Readers. If there is a constant stream of incoming readers, a writer may never get a chance to acquire the lock because the read_count never reaches zero.*
- *Linux 3.16 – replaced rwlock_t by Queued Read-Write lock. Writer also enters the queue making it fair for both.*
- *RCU (Read-Copy-Update): For very high performance (like in the network stack), Linux uses RCU. Readers never wait and don't even take a lock. Writers create a copy of the data, update it, and swap the pointer. The old version is deleted only when all readers are done.*

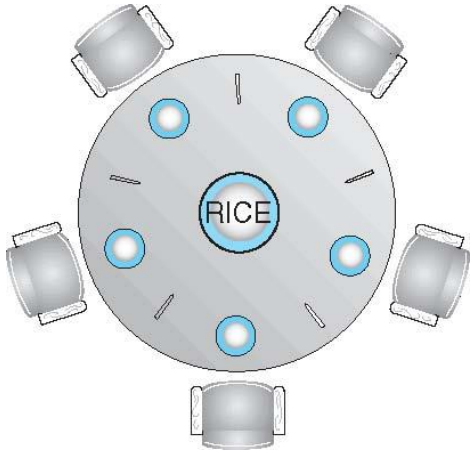
OPERATING SYSTEMS

Dining-Philosophers Problem



PES UNIVERSITY
CELEBRATING 50 YEARS

Dining-Philosophers Problem



- Philosophers spend their lives alternating thinking and eating.
- Don’t interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl.
  - Need both to eat, then release both when done.
- In the case of 5 philosophers
  - Shared data
    - Bowl of rice (data set)
    - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem Algorithm

- The structure of Philosopher *i*:

```
do {
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );
            //  eat
    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );
            //  think
} while (TRUE);
```

- What is the problem with this algorithm?

PES UNIVERSITY
CELEBRATING 50 YEARS

Dining-Philosophers Problem Algorithm (Cont.)



- Deadlock handling
  - Allow at most 4 philosophers to be sitting simultaneously at the table.
  - Allow a philosopher to pick up the chopsticks only if both are available (picking must be done in a critical section).
  - Use an asymmetric solution -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.

Real life examples



- *Database Transactions (Multi-table Updates): If Transaction A locks Table_1 and needs Table_2, while Transaction B locks Table_2 and needs Table_1, a deadlock occurs.*

- *Resource Allocation in Distributed Systems: In a network, a task might need a specific scanner and a specific printer to proceed. If two tasks each grab one and wait for the other, the system hangs.*

- *Parallel Computing: In high-performance computing, different threads may need access to multiple shared memory segments or hardware accelerators (like multiple GPUs) to complete a single computation block.*



THANK YOU

Dr. Anitha K

Department of Computer Science and Engineering

anithak@pes.edu